STRIPE PROJECT

2. OLAP Data System

D8.2. OLAP SQL Queries Examples

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / D8 : OLAP Queries / v08 - 2026/10/13.*

D8.2.1. Contexte

Le présent document propose quelques exemples de requêtes sur les données OLAP pour chacun des processus métiers (cf. ci-dessous). L'objectif est de vérifier que chaque processus est exploitable sans jointure OLTP.

Processus métiers cibles (rappel)

- P1 : Traitement d'un paiement
- P2 : Évaluation du risque de fraude
- P3 : Cycle de vie d'un abonnement
- P4 : Accès et modification de données
- P5 : Traitement d'une demande de droit
- P6 : Traitement d'un incident de sécurité

D8.2.2. P1 : Traitement d'un paiement

Données utilisées : les faits `FactTransaction` & les aggrégats `AggRevenueDailyMerchant`.

P1.1. : Revenus mensuels nets des marchands vs. commission *Stripe*

Récupérer les revenus mensuels nets des marchands (dans la devise de référence) et les commissions Stripe correspondantes, sur les 12 derniers mois.

```sql
SELECT
    d.year,
    d.month,
    m.legal_name AS merchant,
    COUNT(*) AS transactions
    SUM(ft.amount_eur) AS transactions_amount_euro,
    SUM(ft.merchant_net_revenue_eur) AS merchant_net_revenue_euro,
    SUM(ft.stripe_revenue_eur) AS stripe_revenue_euro,
FROM FactTransaction ft
JOIN DimDate d ON d.date_key = ft.date_key
JOIN DimMerchant m ON m.merchant_key = ft.merchant_key AND m.is_current = true
WHERE d.full_date >= (CURRENT_DATE - INTERVAL '12 months')
GROUP BY d.year, d.month, m.legal_name
ORDER BY d.year, d.month, merchant_net_revenue_euro DESC;
```

P1.2 : Top-10 des produits par revenu (tous marchands confondus)

Récupérer les 10 produits qui rapportent le plus (dans la monnaie de référence).

```sql
SELECT
    p.name                    AS product,
    COUNT(*)                  AS transactions,
    c.currency_code           AS transaction_currency,
    SUM(ft.amount)            AS amount_in_transaction_currency,
    SUM(ft.amount_eur)        AS amount_in_euros
FROM FactTransaction ft
JOIN DimProduct p  ON p.product_key = ft.product_key
JOIN DimCurrency c ON c.currency_key = ft.currency_key
```

```
GROUP BY p.name, c.currency_code
ORDER BY amount_in_euros DESC
LIMIT 10;
```

P1.3 : Tableau de bord des revenus quotidiens des marchands

Récupérer les indicateurs de revenus quotidiens des marchands (dans la monnaie de référence), sur les 30 derniers jours.

> **Glossaire**
>
> - *Gross Volume* = montant brut des transactions (pour un jour et un marchand donné)
> - *Net Revenue* = revenu net du marchand (pour un jour et un marchand donné)

```
SELECT
    d.full_date AS date,
    m.legal_name AS merchant,
    a.transaction_count AS transactions,
    a.gross_volume_eur AS gross_volume_in_euros,
    a.merchant_net_revenue_eur AS net_revenue_in_euros,
    a.avg_ticket_eur AS average_ticket_in_euros,
FROM AggRevenueDailyMerchant a
JOIN DimDate d ON d.date_key = a.date_key
JOIN DimMerchant m ON m.merchant_key = a.merchant_key AND m.is_current = true
WHERE d.full_date >= CURRENT_DATE - INTERVAL '30 days'
ORDER BY d.full_date DESC, a.gross_volume_eur DESC;
```

D8.2.3. P2 : Évaluation du risque de fraude

Données : faits `FactFraudEvent`.

P2.1 : Taux de blocage automatique par marchand

Récupérer les taux de transactions bloquées des marchands.

```
SELECT
    m.legal_name AS merchant,
    COUNT(*) AS fraud_events,
    COUNT(*) FILTER (WHERE frl.blocks_transaction) AS blockings,
    ROUND(100.0 * COUNT(*) FILTER (WHERE frl.blocks_transaction)
          / NULLIF(COUNT(*), 0), 2) AS blockings_percentage
FROM FactFraudEvent ffe
JOIN DimMerchant m ON m.merchant_key = ffe.merchant_key AND m.is_current = true
JOIN DimFraudRiskLevel frl ON frl.risk_level_key = ffe.risk_level_key
GROUP BY m.legal_name
ORDER BY blockings_percentage DESC;
```

P2.2 : Taux de fraude confirmée par niveau de risque évalué

Récupérer les taux de fraudes confirmées par niveaux de risque.

```
SELECT
    frl.label AS risk_level,
    COUNT(*) AS fraud_events,
    COUNT(*) FILTER (WHERE ffe.is_confirmed_fraud) AS confirmed_frauds,
    ROUND(100.0 * COUNT(*) FILTER (WHERE ffe.is_confirmed_fraud)
          / NULLIF(COUNT(*), 0), 3) AS confirmation_rate_percentage
FROM FactFraudEvent ffe
JOIN DimFraudRiskLevel frl ON frl.risk_level_key = ffe.risk_level_key
GROUP BY frl.label
ORDER BY confirmation_rate_percentage DESC;
```

P2.3 : Montant exposé par niveau de risque et par région

Récupérer les montants exposés par niveaux de risque et par région.

```sql
SELECT
    g.region,
    frl.label AS risk_level,
    SUM(ffe.exposed_amount_eur) AS exposed_amount_in_euros,
    COUNT(*) AS assessments
FROM FactFraudEvent ffe
JOIN DimGeography g ON g.geography_key = ffe.geography_key
JOIN DimFraudRiskLevel frl ON frl.risk_level_key = ffe.risk_level_key
GROUP BY g.region, frl.label
ORDER BY exposed_amount_in_euros DESC;
```

D8.2.4. P3 : Cycle de vie d'un abonnement

Données : faits `FactSubscriptionSnapshot` & agrégats `AggSubscriptionDailyMerchant`.

P3.1. Derniers revenus récurrents mensuels par marchand

Calculer la totalité des revenus récurrents mensuels des marchands rapportés par les abonnements.

```sql
WITH most_recent_snapshot AS (
    SELECT DISTINCT ON (subscription_id)
        subscription_id, merchant_key, mrr_eur, stripe_mrr_eur, is_active
    FROM FactSubscriptionSnapshot
    ORDER BY subscription_id, date_key DESC
)
SELECT
    m.legal_name AS merchant,
    COUNT(*) FILTER (WHERE mrs.is_active) AS active_subscriptions,
    SUM(mrs.mrr_eur) FILTER (WHERE mrs.is_active) AS merchant_mrr_in_euros,
    SUM(mrs.stripe_mrr_eur) FILTER (WHERE mrs.is_active) AS stripe_mrr_in_euros
FROM most_recent_snapshot mrs
JOIN DimMerchant m ON m.merchant_key = mrs.merchant_key AND m.is_current = true
GROUP BY m.legal_name
ORDER BY mrr_merchant_eur DESC;
```

P3.2. Abonnements à risque de désabonnement par segment

Récupérer le nombre d'abonnements à risque de désabonnement par segment.

```sql
SELECT
    c.segment,
    COUNT(DISTINCT fss.subscription_id) AS dunning_subscriptions,
    AVG(fss.failed_attempt_count) AS average_failed_attempts
FROM FactSubscriptionSnapshot fss
JOIN DimCustomer c ON c.customer_key = fss.customer_key AND c.is_current = true
JOIN DimSubscriptionStatus ss ON ss.subscription_status_key = fss.subscription_status_key
WHERE ss.triggers_dunning = true
  AND fss.date_key = (SELECT MAX(date_key) FROM FactSubscriptionSnapshot)
GROUP BY c.segment
ORDER BY dunning_subscriptions DESC;
```

P3.3. Évolution mensuelle du taux de désabonnement

Récupérer les taux de désabonnement mensuels des marchands.

```sql
SELECT
    d.year,
    d.month,
    m.legal_name AS merchant,
    SUM(a.churned_count) AS total_unsuscribes,
    AVG(a.churn_rate) AS average_unsuscribes_rate
```

```
FROM AggSubscriptionDailyMerchant a
JOIN DimDate d     ON d.date_key = a.date_key
JOIN DimMerchant m ON m.merchant_key = a.merchant_key AND m.is_current = true
GROUP BY d.year, d.month, m.legal_name
ORDER BY d.year, d.month;
```

D8.2.5. P4 : Accès et modification de données

Données : faits `FactAuditEvent` et agrégats `AggComplianceDailyMerchant`.

P4.1. Tentatives d'accès non autorisées par acteur

Récupérer le nombre de tentatives d'accès non autorisés de chaque acteur sur les 7 derniers jours.

```
SELECT
    act.actor_id,
    act.actor_type,
    COUNT(*) AS unauthorized_attempts,
    MAX(fae.event_timestamp) AS last_event_timestamp
FROM FactAuditEvent fae
JOIN DimActor act ON act.actor_key = fae.actor_key AND act.is_current = true
JOIN DimDate d    ON d.date_key = fae.date_key
WHERE fae.was_authorized = false
  AND d.full_date >= CURRENT_DATE - INTERVAL '7 days'
GROUP BY act.actor_id, act.actor_type
ORDER BY unauthorized_attempts DESC;
```

P4.2. Répartition des accès sensibles par marchand

Récupérer les taux des accès sensibles des marchands sur les 30 derniers jours.

> **Glossaire**
>
> - *Sensitive Access* = accès sensibles à des données transactionnelles.
> - *Export* = exportation de données transactionnelles (fichier csv, etc.).

```
SELECT
    m.legal_name AS merchant,
    d.full_date AS date,
    a.total_access_count,
    a.sensitive_access_count,
    a.export_count,
    ROUND(100.0 * a.sensitive_access_count / NULLIF(a.total_access_count, 0), 2) AS
sensitive_rate_percentage
FROM AggComplianceDailyMerchant a
JOIN DimMerchant m ON m.merchant_key = a.merchant_key AND m.is_current = true
JOIN DimDate d ON d.date_key = a.date_key
WHERE d.full_date >= CURRENT_DATE - INTERVAL '30 days'
ORDER BY sensitive_rate_percentage DESC;
```

P4.3. Historique d'accès à un client donné

Récupérer l'historique des tentatives d'accès des acteurs pour un client donné (`:customer`).

```
SELECT
    fae.event_timestamp,
    act.actor_id,
    aa.label AS action,
    fae.resource_type,
    fae.was_authorized
FROM FactAuditEvent fae
JOIN DimCustomer c ON c.customer_key = fae.customer_key
JOIN DimActor act ON act.actor_key = fae.actor_key
JOIN DimAuditAction aa ON aa.audit_action_key = fae.audit_action_key
```

```
WHERE c.customer_id = :customer_id
ORDER BY fae.event_timestamp DESC;
```

D8.2.6. P5 : Traitement d'une demande de modification

Données : faits `FactDataSubjectRequest`.

P5.1. Taux de respect des délais légaux par réglementation

```
SELECT
    r.label AS reglementation,
    r.response_deadline_days,
    COUNT(*) AS requests,
    COUNT(*) FILTER (WHERE fdsr.is_within_deadline) AS requests_within_deadline,
    ROUND(100.0 * COUNT(*) FILTER (WHERE fdsr.is_within_deadline)
          / NULLIF(COUNT(*), 0), 2) AS compliance_rate_percentage
FROM FactDataSubjectRequest fdsr
JOIN DimRegulation r ON r.regulation_key = fdsr.regulation_key
GROUP BY r.label, r.response_deadline_days
ORDER BY compliance_rate_percentage ASC;
```

P5.2. Délai moyen de traitement par type de demande

```
SELECT
    fdsr.request_type,
    COUNT(*) AS fulfilled_requests,
    ROUND(AVG(fdsr.processing_days), 1) AS average_processing_time_days,
    MAX(fdsr.processing_days) AS max_processing_time_days
FROM FactDataSubjectRequest fdsr
JOIN DimRequestStatus rs ON rs.request_status_key = fdsr.request_status_key
WHERE rs.is_terminal = true
GROUP BY fdsr.request_type
ORDER BY average_processing_time_days DESC;
```

P5.3. Demandes en cours proches de l'échéance légale

```
SELECT
    m.legal_name AS merchant,
    fdsr.request_id,
    fdsr.request_type,
    d.full_date AS reception_date,
    d.full_date + (r.response_deadline_days || ' days')::interval AS legal_deadline_date,
    (d.full_date + (r.response_deadline_days || ' days')::interval)::date - CURRENT_DATE AS
remaining_days
FROM FactDataSubjectRequest fdsr
JOIN DimDate d ON d.date_key = fdsr.received_date_key
JOIN DimRegulation r ON r.regulation_key = fdsr.regulation_key
JOIN DimRequestStatus rs ON rs.request_status_key = fdsr.request_status_key
JOIN DimMerchant m ON m.merchant_key = fdsr.merchant_key AND m.is_current = true
WHERE rs.is_terminal = false
ORDER BY remaining_days ASC;
```

D8.2.7. P6 : Traitement d'un incident de sécurité

Données : faits `FactSecurityIncident`.

P6.1. Délai de notification moyen par sévérité

```
SELECT
    fsi.severity,
    COUNT(*) AS incidents,
    COUNT(*) FILTER (WHERE fsi.is_within_72h_deadline) AS incidents_within_deadline,
    ROUND(AVG(fsi.detection_to_notification_hours), 1) AS average_notification_time_hours
```

```sql
FROM FactSecurityIncident fsi
WHERE fsi.notified_date_key IS NOT NULL
GROUP BY fsi.severity
ORDER BY average_notification_time_hours DESC;
```

P6.2. Enregistrements affectés par marchand et par type d'incident

```sql
SELECT
    COALESCE(m.legal_name, 'transversal platform') AS merchant,
    fsi.incident_type,
    COUNT(*) AS incidents,
    SUM(fsi.affected_record_count) AS total_affected_records
FROM FactSecurityIncident fsi
LEFT JOIN DimMerchant m ON m.merchant_key = fsi.merchant_key AND m.is_current = true
GROUP BY merchant, fsi.incident_type
ORDER BY total_affected_records DESC;
```

P6.3. Incidents non résolus par ancienneté et par sévérité

```sql
SELECT
    fsi.incident_id,
    COALESCE(m.legal_name, 'transversal platform') AS merchant,
    fsi.severity,
    d.full_date AS detection_date,
    CURRENT_DATE - d.full_date AS open_days
FROM FactSecurityIncident fsi
JOIN DimDate d              ON d.date_key = fsi.detected_date_key
JOIN DimIncidentStatus ist  ON ist.incident_status_key = fsi.incident_status_key
LEFT JOIN DimMerchant m     ON m.merchant_key = fsi.merchant_key AND m.is_current = true
WHERE ist.is_terminal = false
ORDER BY fsi.severity DESC, open_days DESC;
```